

🔗 Generare una Frase Seme BIP39 con un Dado a 20 Facce — Edizione Estesa: 12 e 24 Parole

Una frase seme mnemonica BIP39 fornisce l'entropia principale del tuo portafoglio Bitcoin. Il software può generare questa casualità al posto tuo, ma i dadi fisici offrono una fonte di entropia trasparente e verificabile, che non dipende da alcun generatore elettronico di numeri casuali. Questa edizione estesa copre la procedura completa a 12 parole, la procedura a 24 parole e la scorciatoia del completamento dell'ultima parola offerta da vari strumenti per semi. Usa la stessa tabella di consultazione (sul retro).

📖 Perché entropia verificabile? — l'incidente di luglio 2026

A luglio 2026, più di mille BTC sono stati svuotati in pochi minuti da portafogli i cui semi erano stati creati da dispositivi Coldcard affetti da un difetto del firmware presente dal 2021: la generazione del seme usava silenziosamente un PRNG software prevedibile invece dell' RNG hardware, fornendo molta meno entropia del promesso. Il produttore ha corretto rapidamente — e il suo stesso avviso ufficiale segnala che i semi generati con 50 o più lanci onesti di dadi non sono mai stati a rischio. La lezione non riguarda un marchio: **ogni entropia elettronica è invisibile all'utente**. Non puoi guardare un chip e vedere se la sua casualità è sana — ma puoi guardare un dado. Con questo tutorial, ogni bit del tuo seme proviene da lanci che verifichi con i tuoi occhi.

📊 Capire la Struttura della Tabella

La tabella di riferimento è organizzata come un sistema di consultazione tridimensionale. Ognuna delle 2048 parole dello standard BIP39 è identificata in modo univoco da tre coordinate, che chiameremo D1, D2 e D3. La prima coordinata (**D1**) seleziona una delle **8 sezioni** della pagina, la seconda (**D2**) seleziona una delle **16 righe** all'interno di quella sezione, e la terza (**D3**) seleziona una delle **16 colonne**. Insieme, questi tre valori identificano esattamente una parola della tabella. Le parole del seme sono sempre quelle dello standard BIP39 in inglese, accettate praticamente da ogni portafoglio.

🎲 La Procedura di Lancio (12 Parole)

Per ognuna delle prime 11 parole della tua frase seme, devi ottenere tre risultati validi dal dado. Lancia il dado e registra il risultato solo se mostra un valore tra 1 e 16 (inclusi). Se il dado mostra 17, 18, 19 o 20, **scarta quel lancio** e rilancia finché non ottieni un risultato valido. Ripeti questo processo finché non hai tre numeri validi (D1, D2, D3) per la parola corrente.

💡 Esempio di Generazione di una Parola

Supponi di stare generando la quinta parola del tuo seme. Lanci il dado e ottieni: 19 (non valido, rilancia), 3 (valido, D1 = 3,4), 1 (valido, D2 = 1), 20 (non valido, rilancia), 11 (valido, D3 = 11). Le tue coordinate sono ((3,4), 1, 11). Vai alla sezione 3,4 della tabella, trova la riga 1 e poi la colonna 11 per ottenere la tua parola: **candy**.

🔪 La 12ª Parola

Per generare l'ultima parola del tuo seme BIP39, ripeti la procedura per le prime 2 coordinate esattamente come fatto per le prime 11 parole, selezionando quindi la sezione e la riga della tabella. A questo punto, esiste esattamente una sola parola nella riga selezionata (su 16 alternative) che formerà un seme BIP39 valido. La colonna dell'ultima parola **non** si determina lanciando il dado, ma per tentativi nell'assistente di inserimento del seme del tuo portafoglio — preferibilmente su un dispositivo *air gapped*: rifiuterà le 15 candidate non valide e accetterà l'unica valida.

💡 Esempio di Generazione dell'Ultima Parola

Supponi che le tue prime 11 parole siano, in ordine: *“never, use, this, example, because, private, key, secret, need, phrase, e true”*. Ora genererai la tua 12ª parola. Lanci il dado e ottieni: 18 (non valido, rilancia), 12 (valido, D1 = 11,12), 9 (valido, D2 = 9). Con ciò, la tua 12ª parola è già determinata: prova una per una le parole della riga 9 della sezione 11,12 e vedrai che la parola della colonna 14, **random**, (e solo quella), insieme alle prime 11, in questo ordine, forma un seme BIP39 valido:

1.never 2.use 3.this 4.example 5.because 6.private 7.key 8.secret 9.need 10.phrase 11.true 12.random. ✔️

Verifica (ad es. in uno strumento di completamento): parole valide della sezione 11,12, una per riga (1–16, in ordine): payment, people, planet, pole, possible, prize, pulp, quality, random, raw, relief, result, return, room, royal, say.

🔪 Scorciatoia: Assistenti con Completamento dell'Ultima Parola

Diversi strumenti per semi completano l'ultima parola al posto tuo: date tutte le parole precedenti, elencano esattamente le candidate che rendono valido il *checksum* — per esempio il flusso *final word* di SeedSigner, il calcolatore dell'ultima parola di SeedPicker, il BIP39 Recoverer offline di Coinplate o la pagina BIP39 di Ian Coleman (usata offline). Per un seme di 12 parole esistono sempre esattamente **128 ultime parole valide: precisamente una in ognuna delle $8 \times 16 = 128$ righe** della tabella. Quindi, invece dei tentativi colonna per colonna: lancia D1 e D2 come al solito e prendi, tra le candidate dell'assistente, **quella che si trova nella tua sezione e riga lanciate**. Stessa parola, stessa entropia, meno digitazione. E non è un caso del nostro esempio: *qualunque* siano le prime 11 parole, fissati D1 e D2, una e una sola parola di quella riga soddisfa il checksum — i test di sanità del repository lo verificano meccanicamente su prefissi casuali.

24 Parole

Per un seme di 24 parole (256 bit di entropia), genera le parole 1–23 esattamente come le prime 11 qui sopra: tre lanci validi ciascuna ($23 \times 11 = 253$ bit). Per la **24ª parola**, **lancia solo D1**, selezionando la sezione (altri 3 bit — 256 in totale). La riga e la colonna restano allora fissate dal *checksum* di 8 bit: **esattamente una parola nell'intera tua sezione di 256 parole** completa un seme valido. Trova in uno di questi due modi:

- **Per tentativi** nell'assistente del portafoglio, lungo la sezione lanciata (fino a 256 tentativi — tedioso, ma completamente offline); oppure
- **Completamento dell'ultima parola**: un assistente come quelli sopra offrirà esattamente **8 candidate valide** — **una per sezione**. Il tuo lancio di D1 sceglie la sezione; prendi la candidata che vi si trova.

Uguualmente garantito, non una coincidenza: *qualunque* siano le prime 23 parole, fissato D1, una e una sola parola di quella sezione soddisfa il checksum — anch'esso verificato a macchina dai test di sanità del repository.

Esempio a 24 Parole: la Frase dell'Esempio, Duplicata

Prendi l'esempio a 12 parole qui sopra e duplicalo per ottenere le parole 1–23; per la 24ª, supponi di lanciare: 17 (non valido, rilancia), 11 (valido, D1 = 11,12). Tra le 8 candidate dell'assistente (o per tentativi lungo la sezione 11,12), esattamente una parola di quella sezione — **polar**, riga 4, colonna 12 — completa un seme valido di 24 parole:

```
never use this example because private key secret need phrase true random
never use this example because private key secret need phrase true polar ✓
```

Nota il sacrificio: la duplicazione non può conservare **random** come 24ª parola, perché il checksum a 8 bit dei 256 bit completi differisce dal checksum a 4 bit che rendeva **random** valida a 12 parole. Il checksum reclama l'ultima parola — e cade su **polar**, l'unica parola della *stessa sezione* (D1 = 11,12) di **random** la cui combinazione con le 23 precedenti lo soddisfa. L'elenco completo delle candidate, una per sezione: **because** (1,2), **critic** (3,4), **fresh** (5,6), **ice** (7,8), **limb** (9,10), **polar** (11,12), **spare** (13,14), **wolf** (15,16).

Riepilogo

- Lancia un dado a 20 facce. **Scarta** i risultati **17, 18, 19 o 20**
- Ogni lancio valido (da 1 a 16 inclusi) specifica una coordinata di una parola
- Ognuna delle **8 sezioni (D1)** corrisponde a **2** possibili risultati validi; ognuna delle **16 righe (D2)** e **16 colonne (D3)**, a **1**
- **12 parole**: 11 parole lanciate per intero e poi solo D1 + D2; **35 lanci validi** producono tutti i **128 bit**. L'ultima parola: per tentativi nella riga, oppure la candidata dell'assistente nella tua riga lanciata (esistono **128** candidate valide, una per riga)
- **24 parole**: 23 parole lanciate per intero e poi solo D1; **70 lanci validi** producono tutti i **256 bit**. L'ultima parola: per tentativi nella sezione, oppure la candidata dell'assistente nella tua sezione lanciata (esistono **8** candidate valide, una per sezione)
- I bit di *checksum* (4 oppure 8) sono sempre compito del portafoglio — mai lanciati

Avvertenze di Sicurezza

- Non usare mai questo esempio qui sopra perché le chiavi private, per restare segrete, hanno bisogno che la loro frase sia veramente casuale
- Esegui questa procedura in un luogo privato. Non fotografare né registrare digitalmente i tuoi lanci o i risultati intermedi. Scrivi la tua frase seme finale su carta e conservala in un luogo sicuro
- Usa gli strumenti di completamento solo su dispositivi *air gapped* fidati — vedono il tuo seme per intero. Per la massima sicurezza, digita la tua frase seme solo su dispositivi *air gapped* fidati e di tua proprietà
- Se sospetti che un tuo seme sia stato generato da software o firmware difettoso (come nell'incidente di luglio 2026), genera un seme nuovo con questo metodo e trasferisci lì i tuoi fondi: **aggiornare il firmware non ripara** un seme debole già esistente

Per Saperne di Più

Questo tutorial è opera di **Yuri da Silva Villas Boas** — autore della BIP-450 (Formosa) e creatore del protocollo Great Wall. Repository ufficiale (sorgenti, PDF in 4 lingue, verifica automatica): github.com/Yuri-SVB/BTC-D20 — oppure scansiona il codice QR a fianco.

Articolo breve: perché un avviso discreto era impossibile — Kerckhoffs e l'incidente di luglio 2026: [posts/kerckhoffs-lemma-coldcard.md](https://posts.kerckhoffs-lemma-coldcard.md) nel repository

Altri tutorial, corsi e comunità: yuri-svb.github.io — e seguici su Twitter/X, Nostr e Telegram

Great Wall: Generare un seme forte è solo il primo passo; proteggerlo da furto e coercizione è il resto. Great Wall è il nostro protocollo di software libero per l'autocustodia resistente alla coercizione: github.com/Yuri-SVB

